

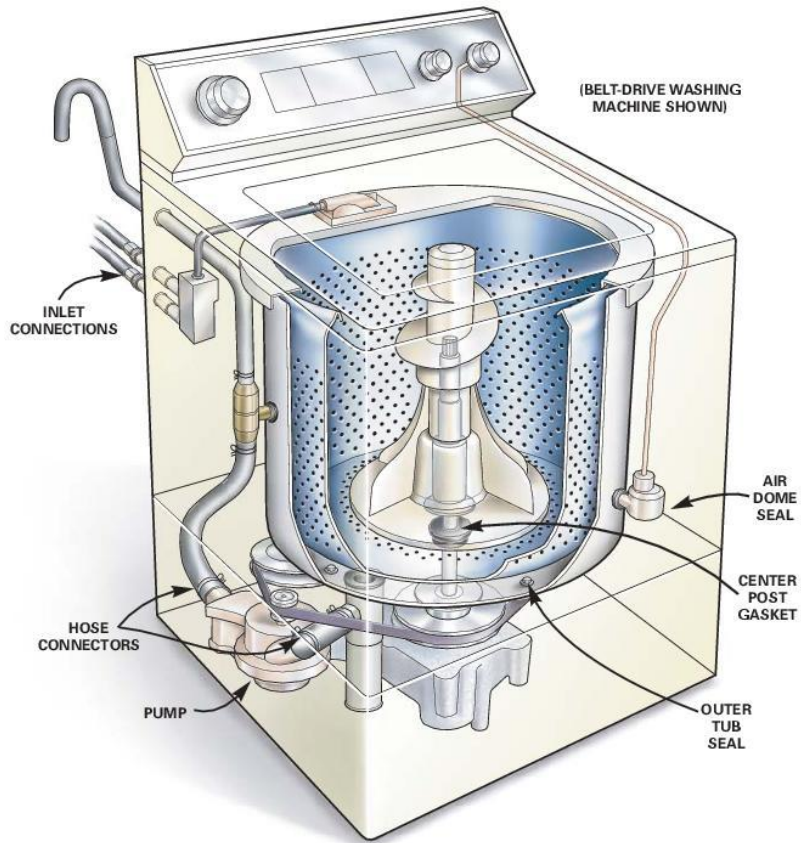
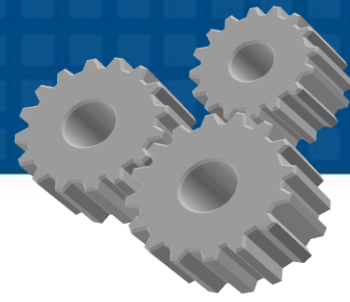


Interface



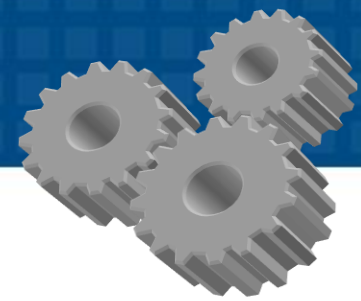
Interface

Khái niệm



Interface

Khái niệm



- Interface định nghĩa một “giao kèo” mà có thể được hiện thực bởi các lớp hay các cấu trúc. Một interface có thể chứa các methods, properties, events, và indexers. Một interface không cung cấp các hiện thực của các thành viên nó định nghĩa.
- Nhận xét
 - Interface chỉ ra làm thế nào các hành vi của lớp/struct được dùng chứ không quan tâm phải làm thế nào phương thức được hiện thực
 - Interface là một lớp trừu tượng thuần túy

Interface

Khai báo

■ Cú pháp

```
[attributes] [modifiers] interface <InterfaceName> [:baseList]
{
    [interface-body]
} [;]
```

Interface

Khai báo

- Tên interface
 - Đặt giống như tên lớp
 - Ký tự đầu tiên là “I”

```
interface IControl
{
    void Paint();
}
```

Interface

Khai báo

■ Danh sách thừa kế

- Interface chỉ có thể thừa kế từ các interface khác
- Interface có thể thực hiện đa thừa kế

```
interface IControl
{
    void Paint();
}
interface ITextBox: IControl
{
    void SetText(string text);
}
interface IListBox: IControl
{
    void SetItems(string[] items);
}
interface IComboBox: ITextBox, IListBox {}
```

Interface

Khai báo

- Các thành phần bên trong interface
 - Phương thức
 - Property
 - Indexer
 - Event
 - Và các thành phần khác được thừa kế từ các interface
- Nhận xét
 - Không có field trong interface
 - Không có constructor và destructor
 - Không có kiểu lồng nhau
 - Không được viết access modifier cho các thành viên

Interface

Hiện thực

- Quy tắc hiện thực interface
 - Một lớp/struct có thể hiện thực **một hay nhiều** interface
 - Khi một lớp cài đặt interface **phải cài đặt mọi thành viên trong các interface** đó
- Có 2 cách hiện thực các thành viên của interface
 - Dùng thành viên **public**
 - Chỉ rõ thành viên thuộc interface nào (tránh tạo thành viên public)

Interface

Hiện thực

■ Cú pháp

```
class ClassName: InterfaceList
{
    1 → public <type> InterfaceMember ()
        {...}
    2 → <type> InterfaceName.InterfaceMember ()
        {...}
}
```

Interface

Hiện thực

```
interface IControl
{
    void Paint();
}
interface IDataBound
{
    void Bind(Binder b);
}
```

■ Cách 1

```
public class TextBox: IControl, IDataBound
{
    public void Paint()
    {
        ...
    }
    public void Bind(Binder b)
    {
        ...
    }
}
```

Interface

Hiện thực

- Khi một lớp hay một struct hiện thực một interface thì các instances của lớp/struct đó có thể được chuyển ngầm định sang kiểu interface đó

```
TextBox editBox = new TextBox();  
editBox.Paint();  
  
IControl control = editBox;  
IDataBound dataBound = editBox;  
control.Paint();  
dataBound.Bind(...);
```

Interface

Hiện thực

```
interface IControl
{
    void Paint();
}
interface IDataBound
{
    void Bind(Binder b);
}
```

■ Cách 2

```
public class TextBox: IControl, IDataBound
{
    public void IControl.Paint()
    {
        ...
    }
    public void IDataBound.Bind(Binder b)
    {
        ...
    }
}
```

Interface

Hiện thực

- Chú ý: các thành viên hiện thực bằng cách 2 chỉ có thể truy cập qua kiểu interface

```
EditText editBox = new  
EditText();  
editBox.Paint();  
  
IControl control = editBox;  
control.Paint();
```

Interface

Hiện thực

```
public interface IFile
{
    int DeleteFile();
    void DisplayFile();
}
public class MyFile : IFile
{
    public int DeleteFile()
    {
        Console.WriteLine("DeleteFile Implementation!");
        return 0;
    }
    public void DisplayFile()
    {
        Console.WriteLine("DisplayFile Implementation!");
    }
}
```

Interface

Hiện thực

```
class InterfaceDemo
{
    public static void Main()
    {
        MyFile objMyFile = new MyFile();
        objMyFile.DisplayFile();
        int retValue = objMyFile.DeleteFile();
    }
}
```

Interface

Hiện thực

- Hiện thực nhiều interface

```
public interface IFileTwo
{
    void ApplySecondInterface() ;
}
```


Interface

Hiện thực

```
public class MyFile: IFile, IFileTwo
{
    public int DeleteFile()
    {
        Console.WriteLine ("DeleteFile Implementation!");
        return (0);
    }
    public void DisplayFile()
    {
        Console.WriteLine ("DisplayFile Implementation!");
    }
    public void ApplySecondInterface()
    {
        Console.WriteLine ("ApplySecondInterface Implementation!");
    }
}
```

Interface

Hiện thực

```
class MultipleInterfaces
{
    static void Main()
    {
        MyFile objMyFile = new MyFile();
        objMyFile.DisplayFile();
        int retValue = objMyFile.DeleteFile();
        objMyFile.ApplySecondInterface();
    }
}
```

Interface

Hiện thực

- Ý nghĩa của cách hiện thực “Chỉ rõ thành viên thuộc interface nào”
 - Ẩn thành viên đối với bên ngoài lớp
 - Tránh trùng tên trong đa thừa kế interface

Abstract class và Interface

Abstract class	Interface
Có thành viên abstract và không abstract	Tất cả thành viên ngầm định là abstract
Định nghĩa lớp abstract có các thành viên abstract = định nghĩa interface	
Có thể có các phần protected, phương thức...	Thành viên của interface là public không có hiện thực
Chỉ được thừa kế từ 1 lớp abstract	Một lớp có thể thừa kế từ 1 hay nhiều interfaces
Lớp abstract có thể thêm nhiều chức năng mà không phá hủy các lớp con	Tạo thêm chức năng sẽ ảnh hưởng đến lớp con

interface-virtual-override-sealed

- **Interface** chỉ ra tên của phương thức
- **Virtual** là implement đầu tiên của phương thức
- **Override** là implement khác của phương thức
- **Sealed** là implement cuối cùng của phương thức

Bài tập

- Dùng phương thức tĩnh `Array.Sort(...)` để sắp xếp các đối tượng của một lớp nào đó
 - Hướng dẫn: Để dùng phương thức `Sort(...)` các phần tử của mảng phải implement giao diện `IComparable`

```
public interface IComparable
{
    int CompareTo(object obj) ;
}
```



Property – Mạng – Indexer



Property

Khái niệm

- Field: Field là một biến thành viên dùng để lưu giữ giá trị của một đối tượng
- Property: Property là mở rộng của field, dùng để cung cấp giá trị mà không cần phải tốn bộ nhớ lưu trữ giá trị (field thông minh – smart field)

Property

Khái niệm

```
class MyClass
{
    public int data;

    public int ABC
    {
        get {}
        set {}
    }
}
```

```
MyClass sp = new MyClass();

// Đọc dữ liệu
x = sp.data;
x = sp.ABC;

// Ghi dữ liệu
sp.data = x;
sp.ABC = x;
```

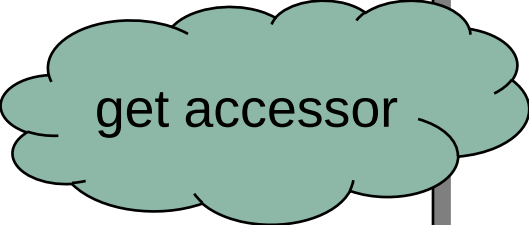
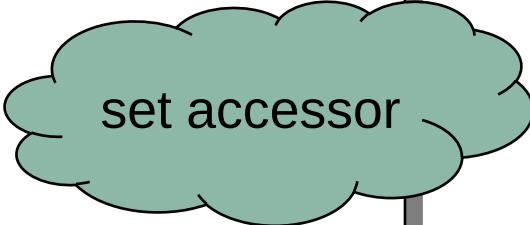
Property

Property và Field

- Giống nhau
 - Cả hai là thành viên có **tên** và có **kiểu**
 - Cú pháp truy cập field và property giống nhau
- Khác nhau
 - Property không giành bộ nhớ chứa dữ liệu
 - Property có các accessor **chứa các câu lệnh** được thực thi khi giá trị của chúng được đọc hay ghi

Property Định nghĩa

■ Cú pháp

```
<modifier> <type> <PropertyName>
{
    get • • •  get accessor
    {
        ...
        return ...;
    }
    set • • •  set accessor
    {
        ...
    }
}
```

Property

Định nghĩa

■ Get accessor

- Được gọi khi bên ngoài đọc dữ liệu của property
- Get accessor tương đương với phương thức không tham số với giá trị trả về thuộc kiểu của property
- Khi property được tham chiếu trong biểu thức thì get accessor được thực thi để tính giá trị của property

■ Set accessor

- Được gọi khi bên ngoài ghi dữ liệu vào property
- Set accessor tương đương với phương thức có một tham số được đặt tên là **value** và không có kiểu trả về
- Khi property được tham chiếu ở bên trái phép gán hay toán tử ++ hoặc -- thì get accessor được thực thi với một tham số là giá trị mới

Property Định nghĩa

```
class MyClass
{
    Random random = new Random();
    public int Data
    {
        get
        {
            Console.WriteLine("GET");
            return random.Next(1, 100);
        }
        set
        {
            Console.WriteLine("SET " + value);
        }
    }
}
```

Property

Định nghĩa

```
class Program
{
    static void Main()
    {
        MyClass obj = new MyClass();
        int x;

        x = obj.Data;
        Console.WriteLine(x);
        obj.Data = x+1;
    }
}
```

Property

Property và Phương thức truy cập

- Phương thức truy cập
 - Để bên ngoài lớp truy cập các field trong một lớp thì lớp đó phải cung cấp các phương thức public: gọi là phương thức Get, phương thức Set

```
class ScreenPosition
{
    private int x, y;

    public int GetX()
    {
        return x;
    }

    public void SetX(int newX)
    {
        x = newX;
    }
}
```

Lấy giá trị
của field

Thay đổi
giá trị của
field

Property

Property và Phương thức truy cập

- Public Property là cách thức khác giúp chúng ta tạo ra cách truy cập các field trong lớp nhưng tự nhiên hơn cho người sử dụng lớp

```
class ScreenPosition
{
    private int x, y;

    public int X
    {
        get { return x; }
        set { x = value; }
    }

    public int Y
    {
        get { return y; }
        set { y = value; }
    }
}
```


Property

Property và Phương thức truy cập

```
class Program
{
    static void Main()
    {
        ScreenPosition pos = new ScreenPosition();
        ...
        oldData = pos.X;
        pos.X = newData;
    }
}
```

Gọi phương
thức getter

Gọi phương
thức setter

- **Read / Write Property:** Cung cấp cả get/set accessor
- **Read - Only Property:** Chỉ cung cấp get accessor
- **Write - Only Property:** Chỉ cung cấp set accessor

Property Phân loại

```
public class Employee
{
    private int empsalary = 15000;
    public int EmpSalary
    {
        get
        {
            return empsalary;
        }
    }
}
```

Property

Phân loại

```
class EmployeeTest
{
    static void Main()
    {
        Employee objEmployee = new Employee();

        Console.WriteLine(
            "Luong nhan vien {0}",
            objEmployee.EmpSalary);

        objEmployee.EmpSalary = 25000;
    }
}
```

Property

Các giới hạn của Property

- Properties không phải là field
 - Không thể khởi tạo property của struct thông qua set accessor
ví dụ: `ScreenPosition location;`
`location.X = 40;`
 - Không thể dùng property làm tham số ref, out
ví dụ: `Method(ref topLeft.X);`
 - Không thể khai báo nhiều property trên 1 câu lệnh
ví dụ: `public int X, Y { get { ... } set { ... } }`
 - Không thể khai báo constant hay readonly property
ví dụ: `const int X { get { ... } set { ... } }`

Property

Các giới hạn của Property

- Properties không phải là phương thức

- Không thể viết code trước get, set

ví dụ:

```
public int X
{
    Console.WriteLine("common code");
    get { ... }          set { ... }
}
```

- Không thể khai báo property kiểu void

```
public void X
{
    get { ... } set { ... }
}
```

Property

- Tương tự các fields và methods, C# hỗ trợ cả instance properties và static properties. Static properties được khai báo với static modifier.
- Các accessor của property có thể là virtual. Khi property khai báo có modifier là virtual, abstract hay override, nó cũng áp dụng cho cả các accessor của property đó.



Mảng



Mảng

Khái niệm

■ Mảng

- Mảng là một cấu trúc dữ liệu chứa một số biến được truy cập thông qua chỉ mục. Các biến trong mảng, cũng được gọi là các phần tử, **có cùng kiểu**

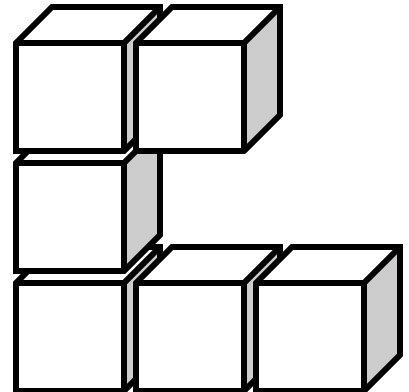
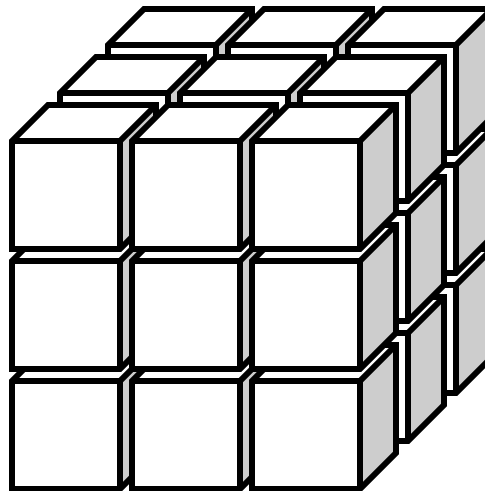
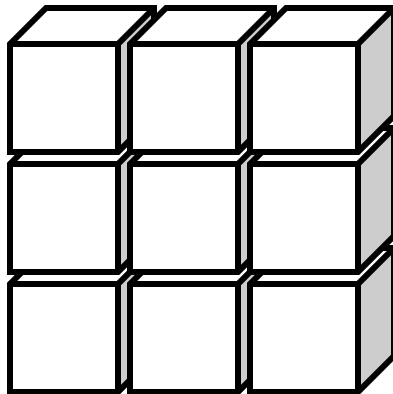
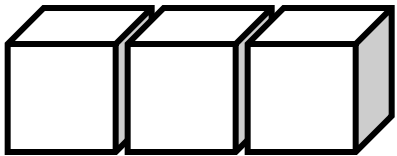
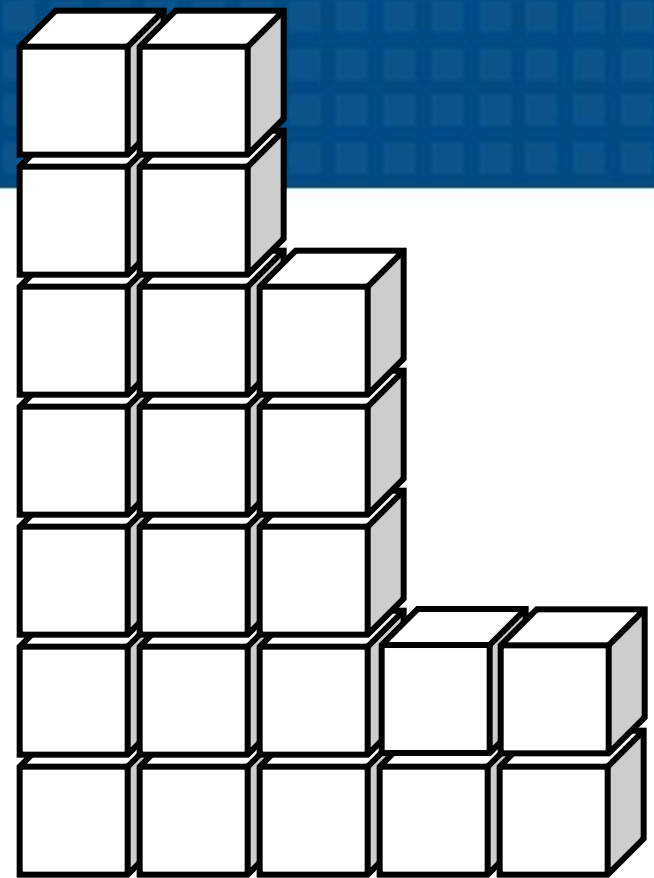
■ Đặc điểm mảng trong C#

- Mảng là **kiểu tham chiếu**
- Tất cả các kiểu mảng đều có lớp cơ sở: `System.Array`

Mảng

Khái niệm

- Phân loại mảng trong C#
 - Mảng một chiều
 - Mảng nhiều chiều
 - Mảng Zích Zắc
 - Mảng hỗn hợp



Mảng

Khai báo

- Mảng 1 chiều

```
type [ ] arrayName;
```

- Mảng nhiều chiều

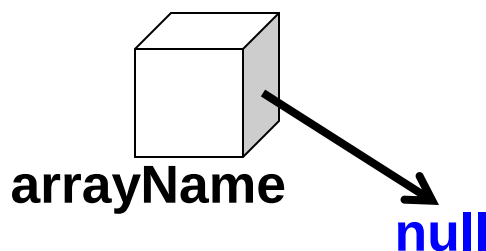
```
type [,] arrayName;  
type [,,] arrayName;
```

- Mảng zích zắc

```
type [ ][ ] arrayName;
```

- Mảng hỗn hợp

```
type [ ][,][,] arrayName;
```



- Chú ý:

- Không chỉ ra kích thước mảng khi khai báo
- Đặt **[]** giữa kiểu dữ liệu và tên biến

Mảng

Bộ khởi tạo

■ Mảng 1 chiều

```
type [ ] arrayName = {v1, v2, ..., vn};  
type [ ] arrayName = new type [ ]{v1, v2, ..., vn};  
type [ ] arrayName = new type [n]{v1, v2, ..., vn};
```

■ Mảng nhiều chiều

```
type [,] arrayName = {  
    {v1,v2,...,vn},  
    ...,  
    {v1,v2,...,vn}};  
type [,] arrayName = new type[,] {...};  
type [,] arrayName = new type[n1, n2] {...};
```

Mảng

Bộ khởi tạo

■ Mảng zich zắc

```
type[ ][ ] arrayName = {  
    new type[ ]{v1,...vn},  
    new type[ ]{v1, ..., vm}, ...};
```

```
type[ ][ ] arrayName = {  
    new type[n]{v1,...vn},  
    new type[m]{v1, ..., vm}, ...};
```

Đúng n
phần tử

```
type[ ][ ] arrayName = new type[ ][ ] {  
    new type[ ]{v1,...vn},  
    new type[ ]{v1, ..., vm}, ...};
```

```
type[ ][ ] arrayName = new type[n][ ] {  
    new type[ ]{v1,...vn},  
    new type[ ]{v1, ..., vm},  
    ...};
```

Đúng n
dòng

Mảng

Truy cập các phần tử

- Mảng 1 chiều

```
arrayName[x];
```

- Mảng nhiều chiều

```
arrayName[x, y];  
arrayName[x, y, z];
```

- Mảng zích zắc

```
arrayName[x][y];
```

- Tạo mảng
 - Dùng bộ khởi tạo
 - Dùng toán tử `new`
- Để lấy **số phần tử** của mảng chúng ta dùng thuộc tính **`arrayName.Length`**

Mảng Tạo

- Mảng 1 chiều

```
arrayName = new type[n];  
type [ ] arrayName = new type [n];
```

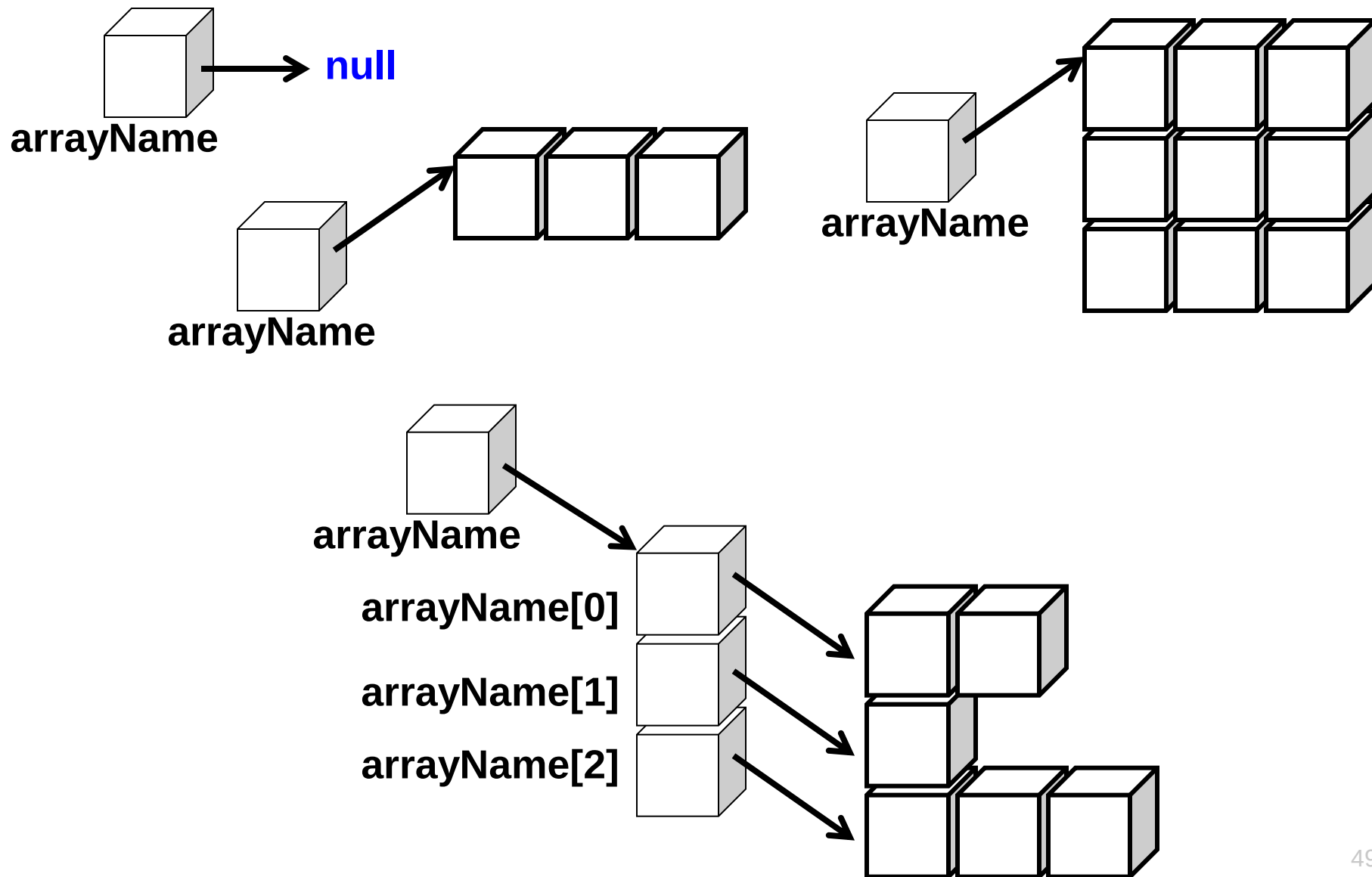
- Mảng nhiều chiều

```
arrayName = new type[n1, n2];  
type [,] arrayName = new type [n1, n2];
```

- Mảng zích zắc

```
type [ ][ ] arrayName = new type [n][ ];  
arrayName[0] = new type[m1];  
arrayName[1] = new type[m2];  
...  
arrayName[n] = new type[mn];
```


Mảng Tạo

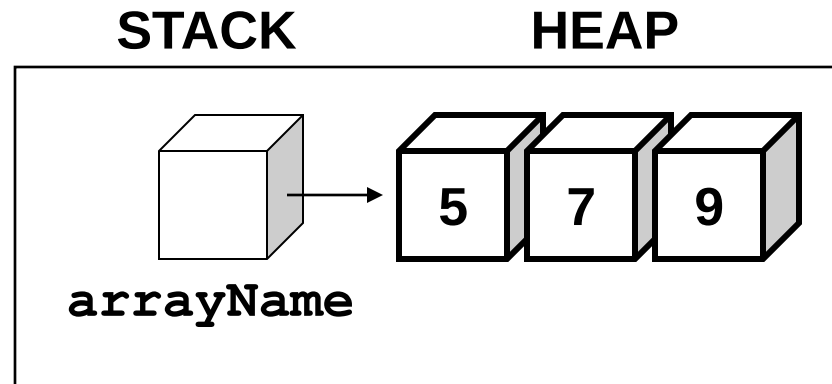


Mảng

Mảng giá trị - Mảng tham chiếu

- Mảng kiểu giá trị

```
int arrayName = new int[3]{5,7,9};
```

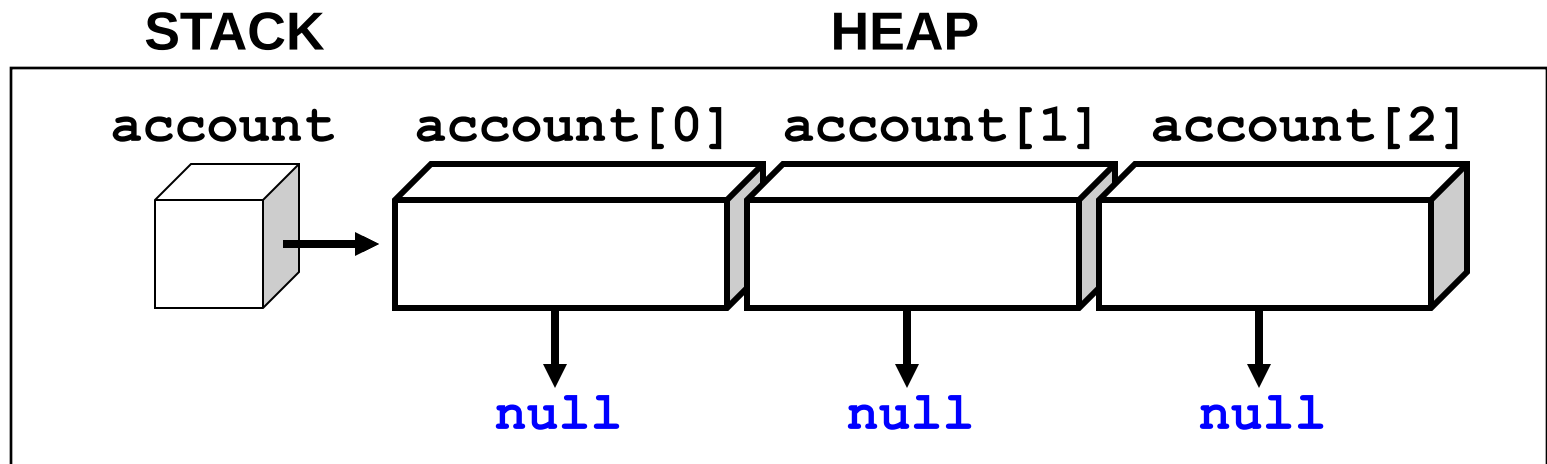


Mảng

Mảng giá trị - Mảng tham chiếu

■ Mảng đối tượng

```
BankAccount account = new BankAccount[3];
```

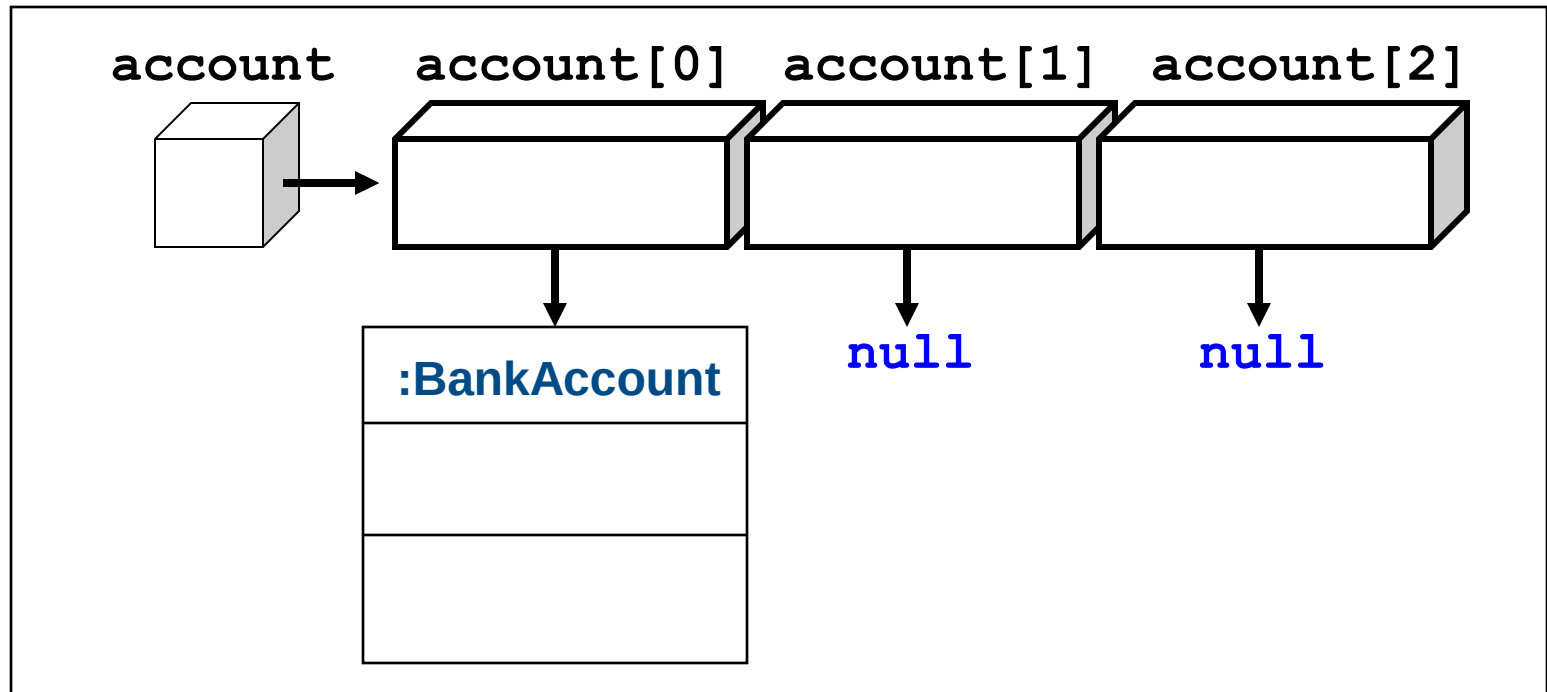


Mảng

Mảng giá trị - Mảng tham chiếu

■ Mảng đối tượng

```
account[0] = new BankAccount();
```



- Nhận xét: Để tạo mảng đối tượng, trước tiên dùng toán tử `new` để tạo mảng các tham chiếu, sau đó lại dùng toán tử để cấp phát các đối tượng mà các phần tử tham chiếu đến

Mảng

Mảng và phương thức

- Kiểu trả về của phương thức là mảng

```
type[ ] MethodName(...)  
{  
    type[ ] result;  
    ...  
    return result;  
}
```

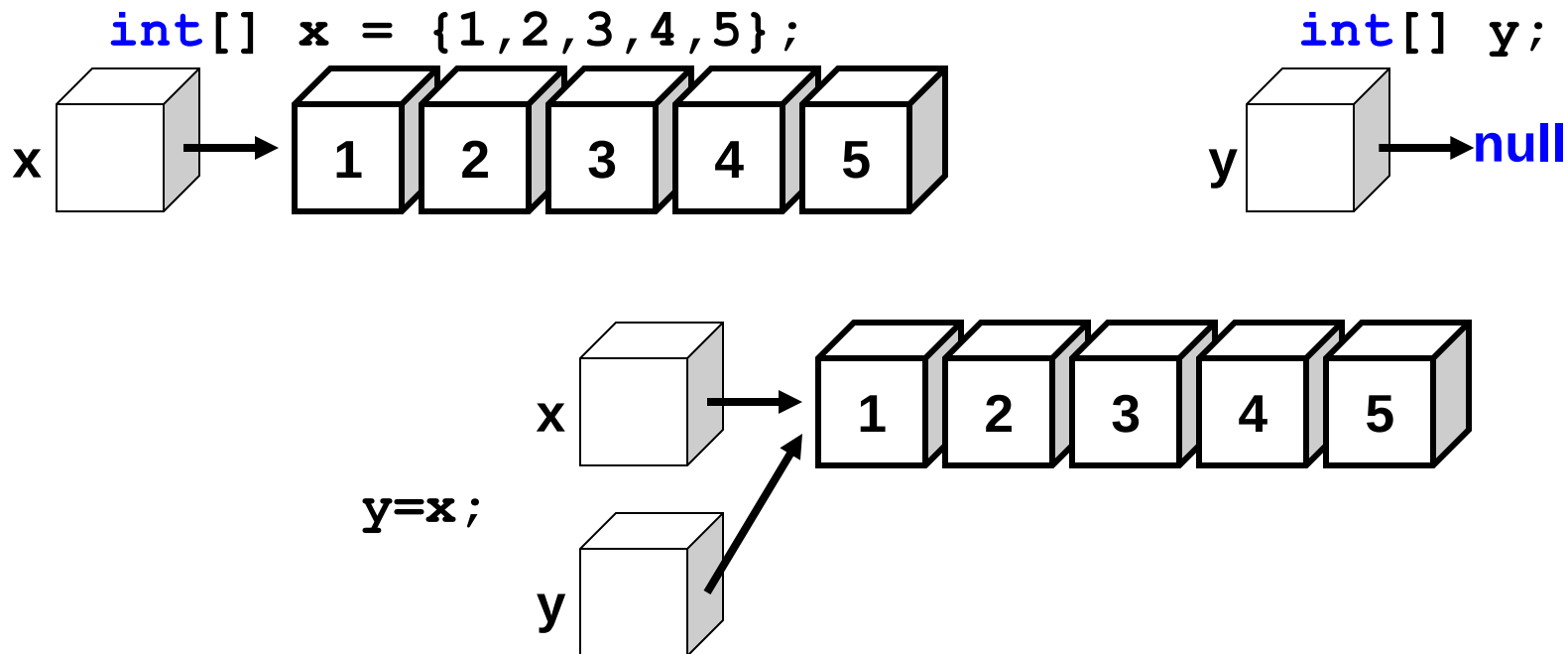
- Tham số của phương thức là mảng

```
MethodName(type[ ] arrayName)  
{  
    ...  
}
```

Mảng

Sao chép Mảng

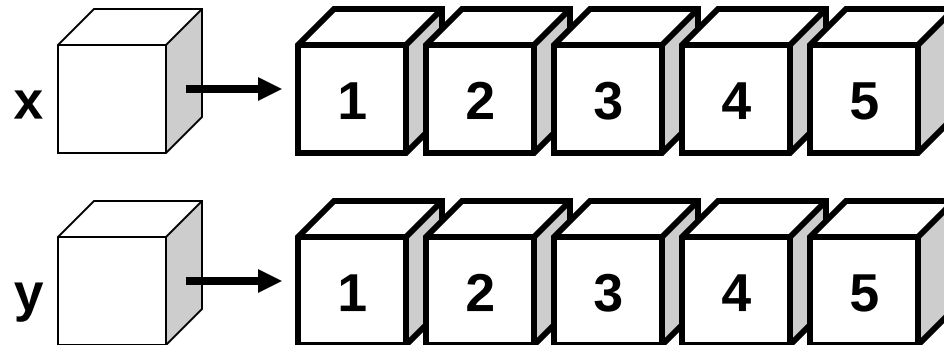
- Mảng là kiểu tham chiếu. Khi sao chép 1 biến mảng sang 1 biến mảng khác, chúng ta sẽ có 2 tham chiếu đến cùng 1 instance mảng



Mảng

Sao chép Mảng

- Nếu muốn sao chép các instance mà mảng đang tham chiếu đến chúng ta thực hiện 2 bước
 - Tạo một instance mảng mới
 - Thiết lập các giá trị trong mảng mới như mảng gốc



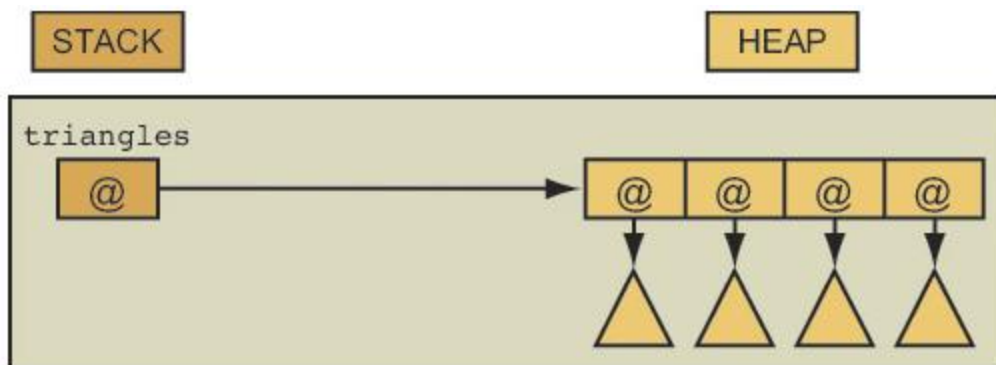
```
int[] y = new int[x.Length];  
  
for (int i=0; i<x.Length; i++)  
    y[i] = x[i]
```

Mảng

Sao chép Mảng

- Sao chép mảng là “shallow copy” chứ không phải “deep copy”

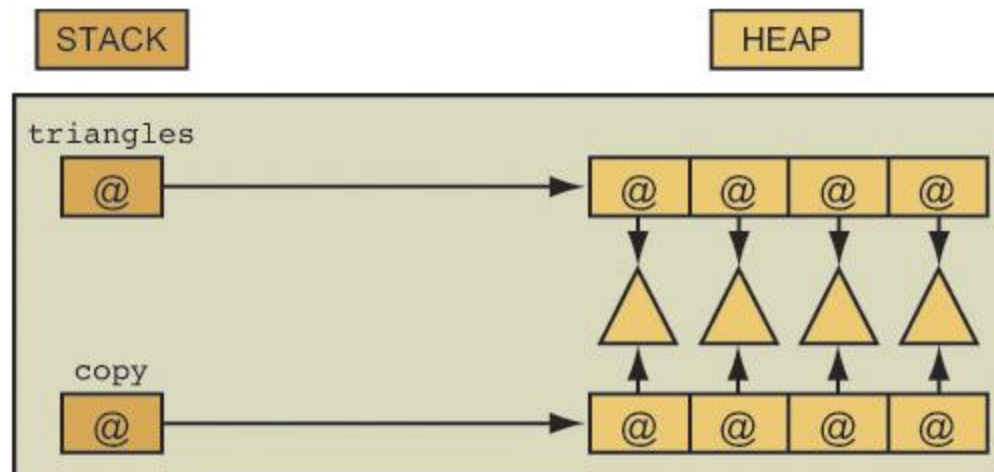
```
Triangle[] triangles = new Triangle[4];  
for (int i = 0; i < triangles.Length; i++)  
{  
    triangles[i] = new Triangle();  
}
```



Mảng

Sao chép Mảng

```
Triangle[] copy = new Triangle[triangles.Length];  
for (int i = 0; i < copy.Length; i++)  
{  
    copy[i] = triangles[i];  
}
```



Mảng System.Array

- Mảng là đối tượng, dẫn xuất từ lớp cơ sở trừu tượng System.Array
- Chúng ta có thể dùng bất kỳ phương thức và thuộc tính nào của System.Array trên mảng do chúng ta tạo ra

Thành viên	Loại thành viên	Ý nghĩa
Rank	Readonly instance property	Trả về số chiều của mảng
GetLength	Instance method	Trả về số phần tử trong chiều được chỉ định của mảng
Length	Readonly instance property	Trả về tổng số phần tử của mảng
GetLowerBound	Instance method	Trả về cận dưới của một chiều cho trước. Phần lớn là 0
GetUpperBound	Instance method	Trả về cận trên của một chiều cho trước. Phần lớn là số phần tử của chiều đó trừ 1
IsReadOnly	Readonly instance property	Cho biết mảng chỉ đọc hay không. Luôn luôn là false
IsSynchronized	Readonly instance property	Cho biết mảng truy cập an toàn thread hay không. Luôn luôn là false
SyncRoot	Readonly instance property	Trả về một đối tượng cho phép truy cập đồng bộ đến mảng. Luôn luôn trả về chính mảng đang dùng
IsFixedSize	Readonly instance property	Cho biết mảng có kích thước cố định hay không. Luôn luôn là true
GetValue	Instance method	Trả về giá trị của phần tử

Mảng System.Array

Thành viên	Loại thành viên	Ý nghĩa
SetValue	Instance method	Thiết lập giá trị của phần tử
GetEnumerator	Instance method	Trả về một IEnumerator. Cho phép dùng câu lệnh foreach
Sort	Static method	Sắp xếp các phần tử trong 1 mảng, trong 2 mảng hay trong một vùng của 1 mảng. Kiểu của các phần tử phải implement IComparer interface hay phải truyền một đối tượng mà kiểu implement interface IComparer
BinarySearch	Static method	Tìm kiếm theo thuật toán tìm kiếm nhị phân. Phương thức này giả sử mảng đã được sắp xếp. Kiểu của các phần tử phải implement IComparer interface. Chúng ta thường dùng phương thức Sort trước khi gọi BinarySearch.
IndexOf	Static method	Trả về vị trí xuất hiện đầu tiên của một giá trị trong mảng 1 chiều hay trong một phần của mảng
LastIndexOf	Static method	Trả về vị trí xuất hiện cuối cùng của một giá trị trong mảng 1 chiều hay trong một phần của mảng
Reverse	Static method	Đảo thứ tự các phần tử trong mảng 1 chiều hay trong một phần của mảng
Clone	Instance method	Tạo một mảng mới là bản sao của mảng gốc
CopyTo	Instance method	Sao chép các phần tử từ một mảng sang một mảng khác
Copy	Static method	Sao chép một phần các phần tử sang mảng khác, thực hiện bất kỳ yêu cầu casting nào
Clear	Static method	Thiết lập một vùng các phần tử thành 0 hay null
CreateInstance	Static method	Tạo một instance của mảng. Phương thức hiếm dùng này cho phép định nghĩa mảng động lúc thời gian chạy với bất kỳ kiểu, vùng, cận
Initialize	Instance method	Gọi constructor mặc nhiên của các phần tử kiểu giá trị (phương thức này không thực hiện nếu các phần tử là kiểu tham chiếu)



Indexer



Indexer

Định nghĩa

- Indexer cho phép xem các đối tượng (có đặc điểm như một mảng) giống như một mảng

```
MyClass obj;  
obj[x] = data;
```

- Cú pháp

```
class ClassName  
{  
    public <type> this [paramater-list]  
    {  
        get { // Return giá trị }  
        set { // thiết lập giá trị mong muốn }  
    }  
}
```

Indexer

Ví dụ

```
class IndexerExample
{
    public string[] stringList = new string[10];
    public string this[int index]
    {
        get
        {
            return stringList[index];
        }
        set
        {
            stringList[index] = value.ToString();
        }
    }
}
```

Indexer

Ví dụ

```
class Test
{
    static void Main()
    {
        IndexerExample indexTest = new IndexerExample();

        indexTest.stringList[1]="Sam";
        indexTest[2]="Tom";

        System.Console.WriteLine(
            "indexTest[1] is {0}\nindexTest[2] is {1}",
            indexTest[1], indexTest[2]);
    }
}
```

Indexer

So sánh Indexer với Phương thức

- Indexer phải có ít nhất 1 tham số

```
public bool this [ ] { ... }
```

- Không thể thêm code ngoài get/set trong indexer

```
public bool this [ int index ]  
{  
    boundsCheck(index) ;  
    get { ... }  
    set { ... }  
}
```

- Indexer không thể có kiểu trả về void

```
public void this [ int index ] { ... }
```


Indexer

So sánh Indexer với Phương thức

- Indexer không trở đến vị trí bộ nhớ
- Indexer có thể có chỉ mục không là số nguyên

```
public int this [string name]  
{ ... } // ok
```

- Indexer có thể được overload

```
public Name this [ PhoneNumber number ]  
{ ... }  
public PhoneNumber this [Name name]  
{ ... } // ok
```

Indexer

Indexer có nhiều tham số

- Nhiều tham số: indexer được truy cập như mảng nhiều chiều

```
...  
class TestMultip  
{  
    public int this[int firstP, int secondP]  
    {  
        //Get and Set Accessors appear here  
    }  
}  
...
```

Indexer

Indexer có nhiều tham số

```
...  
{  
    void static Main()  
    {  
        TestMultip myTest = new TestMultip();  
        int I = myTest[1,1];  
    }  
}
```



Struct và Enum



Struct

- Struct: Struct là một cấu trúc dữ liệu có thể chứa các thành viên dữ liệu và các phương thức. Struct được coi như là một phiên bản lightweight của class
- Đặc điểm của struct
 - Struct là một kiểu giá trị
 - Không thể thừa kế
 - Constructor nếu khai báo rõ ràng thì constructor đó phải có ít nhất 1 tham số
 - Các field không được khởi tạo khi khai báo (khởi tạo trong constructor hay code sử dụng struct)

Struct

■ Cú pháp

```
[attributes] [modifiers] struct <StructName> [: interfaces]  
{  
    [struct-body]  
} [;]
```

Struct

```
struct RGB
{
    public int Red;
    public int Green;
    public int Blue;
}
```

```
RGB rgb;
rgb.Red = 0xFF;
rgb.Green = 0xFF;
rgb.Blue = 0xFF;
```

Struct

■ Sử dụng struct

- Struct thường được dùng cho các cấu trúc dữ liệu nhỏ mà có ngữ nghĩa là giá trị ví dụ: **Complex**, **Point**, các cặp value-key trong dictionary
- Sử dụng struct thay cho class đối với các cấu trúc dữ liệu nhỏ có thể tạo khác biệt lớn về số lượng bộ nhớ được sử dụng

Struct

```
class Point
{
    public int x, y
    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

```
class Test
{
    static void Main()
    {
        Point[] points = new Point[100];
        for (int i = 0; i < 100; i++)
            points[i] = new Point(i, i);
    }
}
```

Struct

```
struct Point
{
    public int x, y;
    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

```
class Test
{
    static void Main()
    {
        Point[] points = new Point[100];
        for (int i = 0; i < 100; i++)
            points[i] = new Point(i, i);
    }
}
```

■ Chú ý

- Không phải khởi tạo struct bằng từ khóa new
- Các thành viên Struct có thể có các constructor, constant, field, method, property, indexer, operator, và các kiểu lồng nhau

Enum

■ Định nghĩa kiểu enum

- Cú pháp

```
enum Name { value1, value2, ... }
```

- Ví dụ

```
enum Color { Red, Green, Blue }
```

- Mặc nhiên, phần tử đầu tiên trong enum được gán giá trị 0 và các phần tử sau có giá trị tăng dần lên 1 (Red=0, Green=1, Blue=2)

Enum

- Giá trị mặc nhiên có thể thay đổi (như ví dụ sau)

```
public enum WeekDays
{
    Monday=1,
    Tuesday=2,
    Wednesday=3,
    Thursday=4,
    Friday=5
}
```

Enum

■ Dùng kiểu enum

- Ví dụ

```
Color color;  
color = Color.Red;
```

- hay

```
color = (Color) 0;
```

■ Hiện giá trị enum

- Ví dụ

```
Console.WriteLine("{0}", color);
```

Enum

- Lặp qua các giá trị của enum

```
foreach (Color color in Enum.GetValues(typeof(Color)))  
{  
    ...  
}
```



Một số lớp/cấu trúc cơ bản



Một số lớp/cấu trúc cơ bản

- Sinh viên hãy tìm hiểu một số thuộc tính và phương thức trong các lớp/cấu trúc sau
 - Convert
 - Math
 - String, Text.StringBuilder
 - Random
 - DateTime
 - Diagnostics.Stopwatch
 - Diagnostics.Process



Collections



Collections

Khái niệm

- Collections chứa các interface và các lớp của một số cấu trúc dữ liệu hữu dụng
- Phân loại Collections
 - **Nongeneric** Collections
 - Namespace: System.Collection
 - Kiểu của các phần tử trong lớp **luôn là object**
 - **Generic** Collections
 - Namespace: System.Collections.Generic

Collections

Khái niệm

■ Một số Nongeneric types

- ArrayList
- Hashtable
- Stack
- Queue
- SortedList

■ Một số Generic types

- List
- Dictionary
- Stack
- Queue
- SortedList
- LinkedList
- SortedDictionary

Collections

ArrayList/List

- **ArrayList/List** là một **mảng động**, cho phép lưu trữ và nhận một chuỗi các giá trị thông qua chỉ mục giống như một mảng
- **ArrayList/List** khắc phục những giới hạn của mảng
 - Kích thước của mảng có thể tự động tăng trưởng khi cần thiết
 - Có thể thêm, xóa phần tử trong mảng dễ dàng

Collections

ArrayList/List

■ Tạo đối tượng

```
ArrayList arrList = new ArrayList();  
ArrayList arrList = new ArrayList(capacity);
```

```
List<type> list = new List<type>();  
List<type> list = new List<type>(capacity);
```

■ Properties

Properties	Ý nghĩa
Capacity	Khả năng
Count	Số lượng phần tử đang chứa

Collections

ArrayList/List

- Thêm dữ liệu

```
listName.Add(data);
```

- Truy cập dữ liệu

```
listName[index];
```

```
foreach (type variable in listName)
{
    xử lý variable
}
```

Collections

ArrayList/List

- Xóa dữ liệu

```
listName.Remove(data);
```

```
listName.Clear();
```

- Chèn dữ liệu

```
listName.Insert(index, data);
```

- Sắp xếp

```
listName.Sort();
```


Collections

ArrayList/List

- Tìm kiếm dữ liệu

```
bool listName.Contains (data) ;
```

```
int listName.IndexOf (data) ;
```

```
int listName.LastIndexOf (data) ;
```

Collections

ArrayList/List

Methods	Ý nghĩa
<code>int Add(object)</code>	Thêm một đối tượng vào cuối mảng
<code>void Clear()</code>	Xóa
<code>bool Contains(object)</code>	Kiểm tra có tồn tại một đối tượng trong mảng
<code>int IndexOf(object)</code>	Tìm kiếm từ trái sang phải
<code>int LastIndexOf(object)</code>	Tìm kiếm từ phải sang trái
<code>void Insert(index, object)</code>	Chèn một đối tượng tại vị trí
<code>void Remove(object)</code>	Xóa phần tử
<code>void RemoveAt(index)</code>	Xóa phần tử tại vị trí
<code>void RemoveRange(index, count)</code>	Xóa một số phần tử
<code>void Sort()</code>	Sắp xếp tăng dần
<code>int BinarySearch(object)</code>	Tìm kiếm nhị phân
<code>void Reverse()</code>	Đảo
<code>IEnumerator GetEnumerator</code>	Trả về IEnumerator

- Chú ý: trong List thay object bằng kiểu tương ứng khi tạo List

Collections

Queue

- Queue hiện thực collection FIFO
 - Phương thức
 - **void** **Enqueue** (**object**) : Thêm vào queue
 - **object** **Dequeue** () Lấy ra khỏi queue
 - **object** **Peek** () : Trả về phần tử đầu queue (nhưng không xóa khỏi queue)
 - Property:
 - **int** **Count**
 - Phương thức khác: **Clear()**, **Clone()**, **Contains()**, **CopyTo()**, **GetEnumerator()**, ...

Collections

Stack

- Stack hiện thực collection LIFO
 - Phương thức
 - **void** `Push(object)` : Thêm vào stack
 - **object** `Pop()` : Lấy ra khỏi stack
 - **object** `Peek()` : Trả về phần tử trên đỉnh stack (nhưng không xóa khỏi stack)
 - Property:
 - **int** `Count`
 - Phương thức khác: `Peek()`, `Clear()`, `Clone()`, `Contains()`, `CopyTo()`, `GetEnumerator()`, ...

Collections

Tự Tạo collection

■ Iteration

- `foreach` hoạt động như thế nào?
- Làm thế nào dùng `foreach` với lớp của chúng ta?

Duyệt các phần tử Collection

```
ArrayList a = new ArrayList();  
a.Add(1); a.Add(5); a.Add(3);  
  
foreach (int x in a)  
{  
    Console.WriteLine(x);  
}
```

```
ArrayList a = new ArrayList();  
a.Add(1); a.Add(5); a.Add(3);  
  
IEnumerator ie = a.GetEnumerator();  
while (ie.MoveNext())  
{  
    int x = (int)ie.Current;  
    Console.WriteLine(x);  
}
```

Collections

Tự Tạo collection

- Câu lệnh foreach làm việc trên collection mà collection thỏa quy tắc sau
 - Hoặc hiện thực interface **IEnumerable**
 - Hoặc hiện thực theo collection pattern:
 - Collection phải có phương thức public GetEnumerator() không tham số và trả về kiểu struct, class hay interface
 - Kiểu trả về của phương thức GetEnumerator() phải chứa
 - Phương thức public MoveNext() không tham số, kiểu trả về bool
 - Phương thức public Current không tham số, kiểu trả về kiểu tham chiếu

Collections

Tự Tạo collection

```
class MyCollectionClass: IEnumerable
{ ... }
```

```
foreach (string s in myCollection)
{
    Console.WriteLine("String is {0}", s);
}
```

```
IEnumerator ie = (IEnumerator) myCollection;
IEnumerator e = ie.GetEnumerator();
```

```
while (e.MoveNext())
{
    string s = (string) e.Current;
    Console.WriteLine("String is {0}", s);
}
```


Collections

Tự Tạo collection

```
public interface IEnumerable
{
    IEnumerator GetEnumerator();
}
```

```
public interface IEnumerator
{
    object Current { get; }
    bool MoveNext();
    void Reset();
}
```

Collections

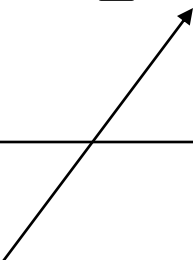
Tự Tạo collection

```
class MyCollectionClass : IEnumerable
{
    CollectionPart[] _parts;
    ...
    ...
    public IEnumerator GetEnumerator()
    {
    }
}
```

Collections

Tự Tạo collection

```
class MyCollectionClass : IEnumerable
{
    CollectionPart[] _parts;
    ...
    ...
    public IEnumerator GetEnumerator()
    {
        return _parts.GetEnumerator()
    }
}
```



Vì `_parts` là `System.Array`, là một kiểu liệt kê nên nó có phương thức `GetEnumerator()` trả về `IEnumerator`

Collections

Tự Tạo collection

- Giải pháp : tiếp cận dựa trên mẫu
 - Trình biên dịch C# compiler tìm kiếm:
 - GetEnumerator() trên collection
 - bool MoveNext() trên kiểu enumerator
 - Current trên kiểu enumerator